PROJECT REPORT: INSURANCE FRAUD DETECTION SYSTEM

1. PROJECT TITLE

FraudShield: An Automated Insurance Fraud Detection System using Machine Learning.

2. PROJECT DESCRIPTION

- **System Overview:** An intelligent software solution designed to analyze historical insurance claim data to identify suspicious patterns and predict the likelihood of fraud.
- **Problem Domain:** Insurance companies face massive financial losses annually due to fraudulent claims. Traditional manual auditing is slow and often misses complex fraud schemes.
- **Core Technology Used:** Python, Scikit-learn, XGBoost, Pandas, and Flask for the web interface.
- **Key Capabilities:** Real-time risk scoring, automated data preprocessing, and interactive dashboards for investigators.
- **Target Users:** Insurance claim adjusters, fraud investigation units, and financial auditors.
- **Real-world Relevance:** Reduces operational costs for insurers and ensures that genuine claims are processed faster by filtering out high-risk cases.

3. APPLICATION SCENARIOS / USE CASES

- **Enterprise Usage:** Integration with existing CRM systems to flag claims during the initial filing stage.
- **End-user Usage:** A web portal where agents can upload claim documents and receive an instant fraud probability score.
- **Industrial Usage:** Analyzing large-scale historical portfolios to detect organized "fraud rings" or recurring patterns.

4. TECHNICAL ARCHITECTURE OVERVIEW

- **High-level Architecture:** User Input -> Data Preprocessing Layer -> ML Model (Inference) -> Result Visualization.
- **Component Interaction:** The Frontend (HTML/CSS) sends JSON data to the Flask API, which loads the pre-trained .pkl model to generate predictions.
- **Data / Request Flow:** Raw claim data flows through a pipeline where categorical variables are encoded, followed by model prediction and database logging.

Explain:

- **Frontend Layer:** Built using HTML5, CSS3, and Bootstrap for a responsive dashboard.
- **Backend Layer:** Python Flask handles the routing and logic.
- **Database Layer:** SQLite/MySQL for storing user credentials and past prediction history.
- **External APIs:** Optional integration with address verification or credit scoring APIs.
- **Deployment Environment:** Hosted on cloud platforms like AWS or Render.

5. PREREQUISITES

5.1 Software Requirements

- **Development Tools:** Python 3.9+
- **IDE:** PyCharm or VS Code.
- **Runtime Environment:** Virtualenv or Docker.
- **Cloud Services:** GitHub for version control.

5.2 Libraries / Frameworks / Dependencies

- pandas & numpy: For data manipulation.
- scikit-learn: For Random Forest and Logistic Regression models.
- xgboost: For high-performance gradient boosting.
- matplotlib & seaborn: For generating analytical graphs.

5.3 Hardware Requirements

- **System Specifications:** Minimum 8GB RAM, Quad-core Processor.
- **Devices / Sensors:** N/A.

6. PRIOR KNOWLEDGE REQUIRED

- **Programming:** Python syntax and data structures.
- **Framework Basics:** Understanding of Flask or FastAPI.
- **Database Fundamentals:** Knowledge of SQL and relational tables.
- **AI/ML Fundamentals:** Concepts of Classification, Overfitting, and Data Imbalance (SMOTE).

7. PROJECT OBJECTIVES

- **Technical:** Achieve an F1-score of over 0.85 in fraud detection.

- **Performance:** Ensure the model processes a single claim in under 500ms.
- **Deployment:** Successfully deploy a working prototype on a web server.
- **Learning:** Master data feature engineering and model hyperparameter tuning.

8. SYSTEM WORKFLOW

1. **User Interaction:** The user enters claim details (e.g., age, claim amount, incident type).
2. **Input Handling:** Data is sanitized and checked for missing values.
3. **Processing Logic:** Features are transformed (One-Hot Encoding, Scaling).
4. **Core System Execution:** The ML model calculates the "Fraud Probability."
5. **Output Generation:** The system classifies the claim as "Legitimate" or "Fraudulent."
6. **Response Delivery:** The UI displays the result with a risk-meter.

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

- **9.1 Problem Definition:** Detecting anomalies in claim behavior using historical trends.
- **9.2 Functional Requirements:** File upload, single-entry prediction, and report generation.
- **9.3 Non-Functional Requirements:** High precision to avoid flagging genuine customers (False Positives).
- **9.4 System Design Decisions:** Using Random Forest to handle non-linear relationships in data.
- **9.5 Technology Stack Selection Justification:** Python is chosen for its extensive libraries and ease of integration.

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

- Installation of Python environment and required libraries via pip.
- Establishment of the directory structure.

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

- **11.1 Feature 1 - Data Engineering:** Converting dates and addresses into usable features.
- **11.2 Feature 2 - Model Training:** Training on the "Insurance Fraud Dataset."
- **11.3 Feature 3 - Web Integration:** Building the API endpoints for the UI.

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

- **Component Integration:** Linking the ML script with the Flask application.
- **Performance Optimization:** Reducing model size using pruning techniques.
- **Security:** Implementing login authentication for the dashboard.

13. MILESTONE 5: TESTING & VALIDATION

- **13.1 Test Cases:** Validated with 20% of the hold-out dataset.
- **13.5 Performance Metrics (AI/ML):**
    - **Accuracy:** 92%
    - **Precision:** 88%
    - **Recall:** 84%
    - **F1 Score:** 86%

14. DEPLOYMENT

- **Hosting Platform:** Render/Heroku.
- **Deployment Steps:** Git Commit -> Push to Main -> Build Environment -> Live.

15. PROJECT STRUCTURE

```
insurance-fraud-detection/
|
├── data/
|   └── processed/
|       ├── cleaned_insurance_data.csv
|       └── insurance_claims.csv
|
├── model/
|   └── best_model.pkl
|
├── src/
|   ├── data_preprocessing.py
|   ├── model_training.py
```

```
│   └── pipeline.py
│
├── templates/
│   ├── index.html
│   └── results.html
│
├── main.py
├── requirements.txt
├── README.md
├── Demo_Link.md
├── .gitignore
└── test.txt
```

16. RESULTS

- The system successfully identifies fraudulent claims based on incident severity and policy age.
- Visual results show a clear distinction between fraud and non-fraud clusters.

17. ADVANTAGES & LIMITATIONS

- **Advantages:** Scalable, reduces human bias, and works 24/7.
- **Limitations:** Requires high-quality labeled data; cannot detect brand-new fraud techniques without retraining.

18. FUTURE ENHANCEMENTS

- **Mobile Integration:** Creating an app for on-site investigators.
- **Automation:** Auto-triggering emails to the legal department upon fraud detection.

19. CONCLUSION

The Insurance Fraud Detection system provides a robust framework for automating risk assessment. By utilizing Machine Learning, companies can protect their assets and provide better services to honest policyholders.

20. APPENDIX

- Source Code: https://github.com/Pallavi1465/Insurance-Fraud-Detection

- Dataset: https://www.kaggle.com/datasets/buntyshah/auto-insurance-claims-data